

Gyro Logic Computational Model

1. 計算モデルの定義

Gyro Logic の計算モデルを C_G とする。

$$C_G = (W, O, Q, P, C, H, G, B, \Gamma)$$

ここで W は World、 O は Observer 集合、 Q は Query、 P は Perception、 C は Conflict、 H は History、 G は Goal、 B は Balance、 Γ は Gyro Step である。

2. 計算状態

計算状態 σ_t を $\sigma_t = (S_t, O_t, H_t, G_t, B_t)$ と定義する。

すなわち、Gyro Logic の1状態は world / observer set / history / goals / balance を含む。

3. Gyro Step

Gyro Logic の基本計算単位を Γ_G とする。

$$\sigma_(t+1) = \Gamma_G(\sigma_t)$$

Gyro Step は Query 生成から再平衡までを1ステップとしてまとめたものになる。

4. Gyro Step の展開

$$\Gamma_G = \text{Rebal} \quad \text{Bal} \quad \text{UpdateGoal} \quad \text{UpdateHistory} \quad \text{Resolve} \quad \text{Detect} \quad \text{Eval} \quad \text{Slice} \\ \text{GenQ}$$

順序としては Generate Query $\rightarrow$ Slice World $\rightarrow$ Evaluate $\rightarrow$ Detect Conflict $\rightarrow$ Resolve Conflict $\rightarrow$ Update History $\rightarrow$ Update Goal $\rightarrow$ Balance $\rightarrow$ Rebalance である。

5. Gyro Machine

Gyro Logic を実行する抽象機械を $GM = (\Sigma, \Gamma_G)$ と定義する。

Σ は状態集合、 Γ_G は状態遷移演算子である。

したがって $\sigma_(t+1) = \Gamma_G(\sigma_t)$ が Gyro Machine の基本遷移式となる。

6. 実行ループ

GyroOS の実行ループは world_flow() $\rightarrow$ generate_queries() $\rightarrow$ slice_world() $\rightarrow$ evaluate() $\rightarrow$ detect_conflicts() $\rightarrow$ resolve_conflicts() $\rightarrow$ update_history() $\rightarrow$ update_goals() $\rightarrow$ evaluate_balance() $\rightarrow$ apply_rebalance() である。

7. メモリモデル

GyroOS のメモリは 3 層で整理できる。

World Memory: 現在の世界状態

History Memory: 観測・衝突・解決の履歴

Observer Memory: 各 Observer の内部記憶・期待・局所 Goal

8. スケジューリング

GyroOS ではスケジューラは Query Scheduler / Observer Scheduler / Conflict Resolver として働く。

従来OSの CPU time slicing より、意味的・観測的な優先順位づけが強い。

9. カーネル対応

GyroOS Kernel は World Engine / Observer Engine / Query Engine / Conflict Engine / Balance Engine の5モジュールに対応する。

10. 命令セット

Gyro Machine の命令セットの最小例：GENQ, SLICE, EVAL, DETECT, MERGE, PRIOR, SUSPEND, HISTORY, GOAL, BALANCE, REBAL

11. 計算の特徴

古典論理が input → deduction → output で記述されるのに対し、Gyro Logic は world → observation → interpretation → conflict → resolution → history → goal → balance → world という循環で記述される。

したがって Gyro Logic は Deductive system ではなく Observation-driven computation である。

12. 要約

Gyro Logic

の計算モデルは、観測・認識差・履歴・目標・再平衡を1つの計算サイクルとして扱う。

これは GyroOS の理論カーネルであり、論理体系とOS設計をつなぐ中間層にあたる。